Inference-Time Recovery of Tool-Use Capability in Low-Bit Quantised Large Language Models

Nitesh Sharma
Independent Researcher

***Abstract*—Small language models (SLMs) are attractive for private, low-cost, and CPU-oriented agent systems, but tool use fails for reasons that are not captured by ordinary language quality metrics. A model may select the wrong tool, emit an invalid call, fail to abstain, or produce one call when a task requires several. We study how much of this failure can be addressed by an inference-time harness rather than by changing model weights. We describe OPENHARN, a small model-agnostic harness with three execution modes: a conventional conversation-based loop, a grammar-constrained text-call mode, and a structured-action mode with externalized state and step verification. The harness also provides schema-derived grammars, grounded filesystem tools, bounded context, text-call recovery, and circuit breakers. On a reproducible 200-example subset of BFCL v4, a 2-bit LFM2 model scores 47.5% with raw native function calling. A prompt-tools configuration initially scores 29.5%, while the later per-request policy reaches 63.0% on the primary full-200 run. A subsequent decomposition fix reaches 64.5% in a later run. On a separate MiniCPM-V quantization experiment, inference-time settings improve Q4 performance from 47.5% to 72.5% on a 40-example category. These results support a narrower claim than "the harness matters more than the model": harnesses can recover mechanical reliability and change the tasks a small model can express, but tool selection, argument values, and task decomposition remain model-limited. We provide a failure taxonomy, limitations, and a pre-registered-style ablation plan for future evaluation.**

I. Introduction

Small language models are increasingly used on local and resource-constrained hardware. Their usefulness as agents, however, depends on more than generating plausible text. An agent must decide whether a tool applies, select the correct tool, emit a valid call, fill its arguments, execute the call, interpret the result, and stop at the right time. These requirements expose failure modes that can be hidden by ordinary text-generation scores.

Quantization and model/server compatibility can make the problem worse. Post-training quantization methods such as GPTQ [1] and AWQ [2] reduce memory and inference cost, but low-bit deployment can expose failures in structured output behavior. A model may know what operation is needed but fail to emit the format expected by the server. Conversely, a grammar can guarantee syntactic validity without making the model choose the right tool or argument values. Treating all failures as "function calling accuracy" obscures this boundary.

This paper asks a focused question: *which tool-use failures can be recovered by an inference-time harness, and which remain limitations of the model?* We study this question using a compact, CPU-oriented implementation rather than a new trained model. The harness is designed for small models and exposes the protocol and execution decisions explicitly enough to inspect them.

Our contributions are:

1) A systems design for small-model tool use that separates caller-supplied tool schemas from built-in handlers and supports native calls, prompt-described grammar-constrained calls, and structured JSON actions.
2) A reliability ladder that combines grounding, schema-derived grammars, text-call recovery, relevance/abstention handling, bounded context, and circuit breakers. These mechanisms address different failure classes rather than acting as a universal configuration.
3) An empirical failure taxonomy distinguishing protocol/mechanical failures from semantic judgment failures, together with measurements on BFCL v4 and a quantization-focused MiniCPM-V experiment.
4) A conservative account of what the current evidence establishes, plus a controlled ablation and held-out evaluation protocol needed to test generality beyond the present runs.

II. Scope and Research Questions

We study inference-time interventions only: no fine-tuning, distillation, preference optimization, or model-specific weight changes. The main questions are:

RQ1: Can a harness improve valid tool-call generation for a quantized SLM?
RQ2: Which interventions help single calls, parallel calls, and abstention, and do those interventions trade off against one another?
RQ3: Does a compact structured-action loop provide a more reliable alternative when a model cannot use native function calling?
RQ4: Which failures remain after syntax and protocol constraints are applied?

The current experiments answer RQ1 and partially answer RQ2 and RQ4. RQ3 is a design contribution and testable hypothesis: the structured SLM mode is implemented in the artifact, but it has not been quantitatively compared with the other modes, so this paper makes no performance claim for it.

III. Related Work

The SLM position paper argues that small models can become useful agents when paired with a good harness [3]. Our

work tests this claim at the tool-call boundary and qualifies it: a harness can improve form and execution discipline, while semantic judgment still depends heavily on the model.

ReAct-style agents maintain a conversational trajectory of thoughts, actions, and observations [4]. Toolformer similarly studies training models to decide when and how to invoke external tools [5]. OPENHARN's default loop follows this inference-time pattern, while its structured mode externalizes durable state and exposes only a small valid action set. This design is related to the harness requirements described by Ranjan and Talluri [6], but our current study does not claim to reproduce their trained specialist model or their end-to-end results.

BFCL v4 provides AST-based function-calling evaluation without executing arbitrary tools [7]. We use its single-turn categories because they isolate call structure, argument values, parallelism, and abstention. AgentBench provides a broader framework for evaluating LLM agents across interactive environments [8], while SWE-bench focuses on issue-resolution performance in real software repositories [9]. We use BFCL because the present study isolates the tool-call layer rather than end-to-end task Natural Language Tools [10], tool-necessity prediction [11], and training-free representation steering [12], together with grammar-guided generation [13], motivate our prompt-tools, YES/NO selection, and schema constraint modes. These mechanisms are established individually; our contribution is their small-model-oriented integration and failure analysis, not the invention of grammar constraints or tool selection.

IV. Harness Design

A. Schema and Handler Separation

Tool schemas are not embedded in the binary. In REPL mode, the caller supplies an OpenAI-style `tools` array through `OPENHARN_TOOLS_SCHEMA`; in server mode, schemas arrive in each request. The harness filters the supplied schemas and derives its prompt and grammar from them. Execution is separate: a requested function name is dispatched to a built-in Rust handler. A schema for an unknown name therefore produces an error rather than silently executing arbitrary code.

The current implementation includes handlers for file reading and writing, anchored editing, project and system search, shell/Python execution, web fetching, and todo state. Existing files must be read in the current session before editing or overwriting. This grounding rule is an execution guard, not a claim that the model understood the whole file.

B. Default Conversational Loop

The default loop maintains a system prompt, user turns, assistant tool calls, and tool results. It sends this history to an OpenAI-compatible endpoint, executes accepted calls, and continues until a text response is produced. The history is trimmed by dropping oldest whole turns, preserving the system message and recent tool context. Tool results are capped to prevent one search from consuming the context.

The loop has operational safeguards: per-turn and total call limits, exact-repeat detection, connection retries, and recovery for structured calls that a server returned as text. Recovery handles formats such as `<tool_call>[...]`; it does not convert arbitrary Markdown shell fences into tool calls.

C. Grammar-Constrained Text Mode

Some model/server combinations do not reliably use the native tool API. With `OPENHARN_PROMPT_TOOLS=1`, tools are described in the system prompt and the native `tools` field is omitted. With `OPENHARN_STRICT_TOOLS=1`, a GBNF grammar restricts output to a schema-valid text call or ordinary text. The grammar is generated from the caller's schemas, so it constrains function names, argument keys, and value types without hardcoding a tool inventory.

A key implementation lesson was bounded whitespace. An unbounded grammar rule allowed a model to emit a valid first call followed by whitespace until the token limit, leaving no closing array delimiter. The parser then discarded an otherwise useful call. Bounding this rule and adding incomplete-array recovery improved the measured BFCL configuration, but the intervention is protocol-specific and should not be treated as a universal model improvement.

D. Structured SLM Mode

With `OPENHARN_SLM=1`, the model does not receive the complete conversational transcript. It receives a compact JSON observation containing the goal, step budget, valid actions, discovered files, previous reads, the last result, and any validation feedback. The action space is deliberately small:

- `SEARCH`: search for candidate files;
- `READ`: inspect a discovered file;
- `ANSWER`: respond after at least one read;
- `ESCALATE`: terminate when the task cannot be completed safely.

Actions are validated before execution, results are checked afterward, and a failed step is retried with localized feedback rather than restarting the entire task. This mode tests a different hypothesis from grammar-constrained tool calling: reducing state and action complexity may be more useful than teaching a weak model a larger function-call protocol.

V. Evaluation Protocol

A. Current Evaluation

The primary reported benchmark is a fixed 200-example subset of BFCL v4: 40 examples each from `simple_python`, `multiple`, `parallel`, `parallel_multiple`, and `irrelevance`. We used this subset because full BFCL runs were impractical during CPU-only iteration: each condition required local inference and external AST evaluation, and the parallel categories were especially expensive to repeat. These are subset results, not BFCL leaderboard scores. The main model is LFM2-8B-A1B-UD-Q2_K_XL, a 2-bit quantization with approximately 1B active MoE parameters. Runs used llama-server on CPU with a 16K context and 12 threads. The BFCL notes report

build and temperature details; run-to-run variation is material, particularly for parallel categories.

A secondary experiment uses MiniCPM-V-4.6 on a 40-example `parallel_multiple` category at Q8 and Q4. It tests whether native template controls can recover quantization degradation without changing weights.

B. Definition of the Per-Request Policy

The term "per-request policy" refers to the category-aware policy implemented by `derive_policy`; it is not a logit router or a learned model. For each request, the harness reads the user text and supplied tool schemas, uses the rule-based `harness_decompose` routine to estimate the number of requested operations, and selects a generation path:

- For a likely single-call request, it prefers native function calling and does not force prompt-tools or a call-count hint.
- For a likely multi-call request, it tries the model's native template with a think-then- call phase, then a plan-first prompt that enumerates operations before emitting a call array, and finally strict prompt-tools with an optional count hint.
- For an irrelevant request, or when the decomposition finds no matching tool, it uses the relevance/abstention path and can return `NO_TOOL` without another model call.

Run D is this policy plus two proxy fixes: flattening BFCL's double-nested message structure before sending it to llama-server, and retrying with flattened prompt-tools plus strict grammar when native function calling returns no call. Run D therefore corresponds to the "per-request policy" row in Table I; it does not mean that every request was forced through the same H2, deduplication, or prompt-tools pipeline.

Run E adds the decomposition correction and fast path that were not present in Run D. The original decomposition code expected OpenAI-wrapped tool objects, whereas BFCL supplied flat objects; consequently it silently estimated zero matching tools. The fix accepts the flat schema, makes the no-match path abstain immediately, and simplifies the relevance-gate prompt with a "when in doubt, say no" instruction. This is the decomposition fix behind the 64.5% run. Incomplete-array recovery and bounded-whitespace grammar are shared parser/grammar components, but they are not what distinguishes Run E from Run D.

C. Fairness and Reproducibility Requirements

Configurations were developed iteratively on the fixed subset; reported numbers are therefore directional rather than unbiased estimates. A rigorous methodology would freeze prompts, schemas, decoding parameters, and model/server versions before scoring held-out examples, and would report repeated runs with confidence intervals. It should also compare each intervention independently and in combination, using metrics such as latency, generated tokens, invalid-call rate, hallucinated-result rate, and final task success.

TABLE I
BFCL V4 RESULTS ON THE FIXED 200-EXAMPLE SUBSET. VALUES ARE PERCENTAGES. THE PRIMARY POLICY RESULT IS RUN D; RUN E REACHED 64.5% AFTER AN ADDITIONAL DECOMPOSITION FIX.

Configuration	Simple	Multiple	Parallel	Parallel-M	Irrel.	Overall
Raw native FC	80.0	82.5	0.0	0.0	75.0	47.5
Prompt + strict	35.0	10.0	5.0	5.0	92.5	29.5
+ abstain + gate	60.0	45.0	2.5	25.0	87.5	44.0
Per-request policy (Run D)	75.0	77.5	52.5	42.5	67.5	63.0

VI. RESULTS

A. BFCL Results

Table I summarizes the results documented in the BFCL research notes. The raw native-function-calling result is the baseline. Prompt-tools plus strict grammar is not automatically better: the initial configuration scored lower because the model's native training and the text protocol competed. The later per-request policy, including message flattening and native-empty fallback, produced 63.0% on the full 200-example run. A subsequent decomposition fix produced 64.5%; we use 63.0% as the primary result because it is the stable run discussed in the research notes.

The main pattern is more informative than the aggregate score. Native calling is competitive for ordinary single calls but produces no correct parallel calls for this model and setup. The text grammar expresses arrays of calls and improves parallel categories, while the relevance gate improves abstention. The same strict prompt-tools configuration can damage models whose native function-calling path is healthy; measurements in the project notes show strong native models falling to zero on the tested parallel-multiple setup under the incompatible strict configuration. Thus the result supports per-model or per-request adaptation, not one universal flag set.

B. Quantization Experiment

Table II reports the separate MiniCPM-V experiment. The Q4 model's score rises from 47.5% with automatic native tool choice to 72.5% when tool choice is required and thinking is disabled. The result is encouraging but narrow: it is one model family, one category, and a small evaluation slice. It demonstrates that inference controls can matter substantially; it does not establish that quantization primarily damages format rather than other capabilities.

TABLE II
MINICPM-V-4.6 QUANTIZATION EXPERIMENT ON 40 PARALLEL-MULTIPLE EXAMPLES.

Configuration	Accuracy	Repeated values	Latency
Q8 native, automatic	82.5%	77.5/85/85	baseline
Q4 native, automatic	47.5%	45/50/47.5	–
Q4 required + no-think	72.5%	72.5/72.5/72.5	not measured

C. What the Current Results Do Not Establish

The present experiments do not establish a universal improvement, a causal ranking of every harness component, or

superiority over trained specialist agents. The 200-example BFCL slice was selected for practical feasibility, and some configurations were tuned against related behavioral cases. The results should therefore be read as evidence for specific mechanisms and failure modes, not as a leaderboard claim.

VII. Failure Taxonomy

We classify failures by the layer at which they occur:

- **Protocol failure.** The model emits a format the server cannot parse, or the grammar/parser loses a call through truncation or runaway whitespace. Bounded grammar and recovery can help.
- **Selection or abstention failure.** The model calls a tool when none applies, or a relevance gate says no when a tool is needed. A gate can improve abstention but introduces false negatives.
- **Argument failure.** The model selects a plausible function but supplies a wrong value, type, enum, or nested argument. A grammar can enforce types, not facts.
- **Planning failure.** The model selects the wrong number of calls, under-decomposes a parallel task, duplicates calls, or stops at the wrong time. This remains primarily model judgment, although prompts and structured state can change the interface.

This taxonomy prevents a common overclaim. Valid JSON is not equivalent to correct tool use, and a higher tool-call score does not necessarily imply better end-to-end coding behavior. The appropriate metrics should report syntax validity, tool selection, argument accuracy, call-count accuracy, abstention, execution success, and final task success separately.

VIII. Discussion

The evidence supports a qualified systems conclusion. Harnesses can recover mechanical reliability: they can constrain syntax, prevent unsafe edits, express multiple calls, repair some truncation, and supply an explicit abstention mechanism. They cannot reliably provide missing semantic knowledge, correct argument values, tool competence, or robust decomposition.

The best configuration is therefore conditional. A model with healthy native function calling should not automatically be forced through prompt-tools and strict grammar. A weak model that cannot emit native calls may benefit from a narrow schema, prompt descriptions, and grammar constraints. A model that can emit compact JSON but not tool calls may be a candidate for the structured SLM mode. The practical design principle is a reliability ladder: add constraints only when measurements show that the corresponding failure exists.

The CPU results also expose a deployment tradeoff. Reasoning tokens can dominate wall-clock latency, especially on local hardware. Although the project notes suggest that disabling thinking can improve throughput, exact latency values were not retained for the MiniCPM-V tabled runs, so we do not claim a measured speedup here. Future runs should report the speed/accuracy tradeoff directly.

A. Limitations and Threats to Validity

- The primary BFCL result is a fixed 200-example subset, not the full leaderboard.
- The main BFCL model is one LFM2 quantization and the secondary experiment is one MiniCPM-V family; broad transfer is untested.
- Configurations were developed iteratively on the fixed subset, so reported numbers are directional rather than unbiased estimates; a full held-out multi-model ablation is future work.
- The measurements depend on llama-server chat templates, grammar behavior, and build versions. Another server may produce different results. Guided-generation implementations also differ in their supported grammars and decoding semantics [13].
- BFCL single-turn AST accuracy is not end-to-end coding success and does not evaluate filesystem safety, multi-turn state, or final answer quality.
- The structured SLM implementation is quantitative evidence only after a direct controlled comparison; its current inclusion is a design contribution and testable hypothesis.
- The artifact contains powerful local handlers such as shell and Python execution. The research evaluation assumes a trusted local environment and does not claim sandbox security.

IX. Conclusion

We presented a small-model-oriented tool-use harness and a framework for analyzing where it helps. Existing measurements show that schema-derived grammar, abstention handling, bounded output, and inference controls can substantially change tool-call reliability without training. They also show that a prompt-tools configuration can hurt a model whose native interface is already working, and that semantic judgment remains a bottleneck after syntax is fixed.

The defensible conclusion is not that a harness is universally more important than a model. It is that harness design is a major, measurable part of small-model agent capability. A good harness can recover form and execution discipline; the model still determines much of the judgment. The next step is the controlled, held-out, multi-model ablation described above.

Acknowledgments

We thank the BFCL team and the Gorilla project for the BFCL v4 datasets and evaluation code, and the llama.cpp community for the OpenAI-compatible inference interface used in the experiments. The reported runs were conducted on consumer CPU-oriented hardware.

References

[1] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *International Conference on Learning Representations*, 2023.

[2] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," in *Proceedings of Machine Learning and Systems*, 2024.

[3] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. Lin, and P. Molchanov, "Small language models are the future of agentic ai," *arXiv preprint arXiv:2506.02153v2*, 2025.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *International Conference on Learning Representations*, 2023.

[5] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, 2023.

[6] I. Ranjan and G. Talluri, "Specialized small-model subagents: Co-designing a fine-tuned slm and a task-specific harness for cost-efficient multi-agent systems," 2026, gitHub: IshaanAyaan/slm-agents, white paper at paper/white_paper.md.

[7] S. Patil, T. Mao, W.-L. Cheng, S. Roosta, S. Ji, X. Zou, A. Adala, P. Kumar, H. Yan *et al.*, "The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models," in *International Conference on Machine Learning (ICML)*, vol. 267. PMLR, 2025.

[8] X. Liu, H. Yu, H. Zhang *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations*, 2024.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" *International Conference on Learning Representations*, 2024.

[10] R. T. Johnson, M. D. Pain, and J. D. West, "Natural language tools: A natural language approach to tool calling in large language agents," *arXiv preprint arXiv:2510.14453*, 2025.

[11] C.-H. Sun, L. Liu, G. Yan, Z. Wang, and T.-W. Weng, "Llm agents already know when to call tools — even without reasoning," *arXiv preprint arXiv:2605.09252*, 2026, also known as Probe&Prefill / When2Tool.

[12] Y. Wang, R. Zhou, R. Fu, S. Cao, H. Zeng, J. Lu *et al.*, "Asa: Training-free representation engineering for tool-calling agents," *arXiv preprint arXiv:2602.04935*, 2026.

[13] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.